

**ANALISADOR LÉXICO DA LINGUAGEM ELGOL COM PLY:** implementação,  
instalação e validação por programas de teste<sup>1</sup>

**LEXICAL ANALYZER FOR THE ELGOL LANGUAGE WITH PLY:** *implementation,*  
*installation and validation through test programs*

André Kroetz Berger  
Arthur Contri Gasperin  
Guilherme Gabriel Rigotti  
Henrique Marques Kayser  
Lucas Borkert Beck  
Rafael Schulz Vaz

**RESUMO**

Este artigo descreve a Etapa 1 do projeto da disciplina de Compiladores da Universidade La Salle: um analisador léxico para Elgol, linguagem definida pelo professor Elgio Schlemer, escrito em Python com a ferramenta PLY (Python Lex-Yacc). O texto apresenta as regras léxicas da linguagem e as decisões que o grupo tomou nos pontos em que o enunciado não era explícito, explica por que a validação de identificadores e números foi feita em código Python e não apenas por expressão regular e documenta o roteiro de instalação publicado no repositório, com a validação feita no Windows 11 e no Debian 13. Também relata a bateria de testes: quatro programas Elgol corretos, escritos para exercitar laços, desvios condicionais, funções, comentários e as palavras `_Z_` e `_NEG_`, e cinco programas com erros intencionais. Os programas corretos geraram 91, 96, 97 e 72 *tokens*, sem nenhum erro léxico, e os programas com erro tiveram todos os problemas apontados com linha, coluna e motivo.

**Palavras-chave:** compiladores; análise léxica; PLY; Python; Elgol.

**ABSTRACT**

This article describes Stage 1 of the Compilers course project at Universidade La Salle: a lexical analyzer for Elgol, a language defined by professor Elgio Schlemer, written in Python with the PLY (Python Lex-Yacc) tool. It presents the lexical rules of the language and the decisions the group made where the assignment was not explicit, explains why identifiers and numbers are validated in Python code rather than by regular expressions alone, and documents the installation guide published in the repository, validated on Windows 11 and Debian 13. It also reports the test suite: four correct Elgol programs, written to exercise loops, conditionals, functions, comments and the `_Z_` and `_NEG_` keywords, and five programs with intentional errors. The correct programs produced 91, 96, 97 and 72 tokens with no lexical errors, and every problem in the faulty programs was reported with line, column and reason.

**Keywords:** compilers; lexical analysis; PLY; Python; Elgol.

---

<sup>1</sup> Trabalho apresentado à disciplina de Compiladores, ministrada pelo professor Elgio Schlemer na Universidade La Salle (Unilasalle), Canoas, RS, em outubro de 2026.

## 1 INTRODUÇÃO

Um compilador não lê um programa de uma vez só. O trabalho é dividido em fases, e a primeira delas, a análise léxica, percorre o texto caractere por caractere e agrupa esses caracteres em unidades que têm significado para a linguagem, os *tokens* (Aho et al., 2008). É nessa fase que um nome de variável mal formado ou um símbolo estranho ao alfabeto da linguagem aparece pela primeira vez como erro.

Na disciplina de Compiladores da Universidade La Salle, o projeto do semestre gira em torno de Elgol, uma linguagem criada pelo professor Elgio Schlemer para a disciplina. O projeto foi dividido em etapas. Antes dele veio o TR4, um estudo sobre ferramentas de análise léxica e sintática no qual o grupo escolheu o PLY (Python Lex-Yacc). A Etapa 1, assunto deste artigo, pede um analisador léxico completo para Elgol. A Etapa 2, ainda por fazer, será o analisador sintático.

Todo o material do grupo está em um repositório público no GitHub (Kayser et al., 2026). Além do código, o repositório traz um README com as instruções de uso e as decisões de projeto, um guia de instalação separado, uma pasta com capturas de tela da instalação em dois sistemas operacionais e vários arquivos de teste. O objetivo deste artigo é reunir esse material em um texto único: descrever como o analisador foi construído, como instalar e executar o projeto, como a instalação foi conferida e quais resultados os programas de teste produziram. Os números apresentados nas tabelas correspondem à versão final do repositório.

O texto está organizado da seguinte forma. A Seção 2 apresenta os conceitos de análise léxica e a ferramenta PLY. A Seção 3 resume as regras léxicas de Elgol e as decisões do grupo. A Seção 4 mostra a organização do repositório e a divisão do trabalho entre os integrantes. A Seção 5 descreve a implementação. A Seção 6 documenta a instalação e a validação no Windows 11 e no Debian 13. A Seção 7 apresenta os programas de teste e seus resultados, a Seção 8 traz a conferência das regras e as observações para a próxima etapa, e a Seção 9 encerra com as considerações finais.

## 2 FUNDAMENTAÇÃO TEÓRICA

### 2.1 Análise léxica

Aho et al. (2008) separam três ideias que costumam ser confundidas. O *token* é a categoria, como identificador ou número. O padrão é a regra que descreve quais sequências de

caracteres pertencem a essa categoria, normalmente escrita como expressão regular. O lexema é a sequência concreta encontrada no texto, como `Resultado` ou `130`. O analisador léxico lê a entrada, encontra lexemas que casam com algum padrão e entrega ao analisador sintático a sequência de *tokens* correspondente, descartando o que não interessa às fases seguintes, como espaços e comentários.

Além de produzir *tokens*, essa fase costuma registrar a posição de cada um no arquivo, para que as mensagens de erro digam onde está o problema, e alimentar a tabela de símbolos, estrutura que guarda os nomes usados no programa e é consultada pelas fases seguintes (Aho et al., 2008). As duas coisas aparecem no analisador descrito neste artigo.

## 2.2 A ferramenta **PLY**

O **PLY** é uma implementação em Python das ferramentas clássicas `lex` e `yacc`, escrita por David Beazley. São dois módulos: o `ply.lex`, para a análise léxica, e o `ply.yacc`, para a sintática, que gera analisadores LALR(1). Ao contrário do `Flex` e do `Bison`, o **PLY** não usa um arquivo de especificação separado nem gera código-fonte. As regras são escritas como variáveis e funções Python, e o analisador é montado durante a execução do programa (Beazley, 2018).

No `ply.lex`, cada *token* é definido por uma variável ou por uma função cujo nome começa com `t_`. Uma função de regra recebe o objeto do *token*, pode consultar e alterar seu tipo e seu valor e pode descartá-lo retornando `None`. Esse detalhe foi decisivo na Etapa 1, como mostra a Seção 5.

A versão usada é a 3.11, publicada no Python Package Index em fevereiro de 2018 e fixada no arquivo `requirements.txt` do projeto (Beazley, 2018). O repositório oficial da ferramenta foi arquivado pelo autor em 21 de dezembro de 2025 (Beazley, 2025), o que significa que não haverá novas correções. Para um trabalho acadêmico isso não tem impacto prático, porque a biblioteca é estável e não depende de nenhum outro pacote, mas o grupo registrou o fato no artigo do TR4 como um ponto a considerar em projetos de longa duração.

## 3 A LINGUAGEM ELGOL E SUAS REGRAS LÉXICAS

### 3.1 Classes de *token*

Elgol tem poucos elementos, mas regras pouco usuais. Não há cadeias de caracteres nem números com casas decimais, e não existe sequer o número zero, que é escrito com a palavra

reservada `_Z_`. Números negativos também não podem ser escritos diretamente e dependem do operador `_NEG_`. Cada comando termina com um ponto final, e a multiplicação é representada pela letra `x`. O Quadro 1 resume as classes de *token* que o analisador reconhece.

Quadro 1 - Classes de token da linguagem Elgol

| Classe              | Token                                                                                                                                 | Regra                                                                                                                                                                              |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Identificador       | ID                                                                                                                                    | Começa com consoante maiúscula, tem só letras, pelo menos 4 caracteres e termina com letra minúscula.                                                                              |
| Nome de função      | FUNCAO                                                                                                                                | Cifrão seguido de um identificador válido, como <code>\$Media</code> .                                                                                                             |
| Número inteiro      | NUMERO                                                                                                                                | Primeiro dígito de 1 a 9, seguido de dígitos. O zero é escrito com <code>_Z_</code> .                                                                                              |
| Palavras reservadas | ELGIO, DECIMAL, ZERO, NEG, EXP, RESTO, ENQUANTO, SE, ENTAO, SENAO, PARA, INICIO, FIM, MAIOR, MENOR, IGUAL, DIFERENTE, MIGUAL, MAIGUAL | Lista fechada de 19 palavras, sensível a maiúsculas e minúsculas. A grafia <code>migual</code> gera <code>MIGUAL</code> e a grafia <code>MIgual</code> gera <code>MAIGUAL</code> . |
| Operadores          | ATRIB, MAIS, MENOS, DIVIDE, VEZES, EXP, RESTO                                                                                         | Os símbolos <code>=</code> , <code>+</code> , <code>-</code> e <code>/</code> , a letra <code>x</code> e as palavras <code>EXP</code> e <code>RESTO</code> .                       |
| Delimitadores       | PONTO, VIRGULA, ABRE_PAR, FECHA_PAR                                                                                                   | O ponto encerra cada comando. A vírgula e os parênteses aparecem na declaração e na chamada de funções.                                                                            |
| Comentário          | descartado                                                                                                                            | O asterisco inicia um comentário que vai até o fim da linha.                                                                                                                       |
| Espaços             | descartados                                                                                                                           | Espaço, tabulação, retorno de carro e quebra de linha.                                                                                                                             |

Fonte: Elaborado pelos autores (2026).

### 3.2 Pontos em aberto e decisões do grupo

O enunciado deixou algumas perguntas sem resposta. Em vez de escolher em silêncio, o grupo registrou cada decisão no README e concentrou todas elas em constantes no início do arquivo `lexico_elgol.py`. Assim, se o professor entender alguma regra de outra forma, a mudança se resume a uma linha de código. O Quadro 2 traz essas decisões.

Quadro 2 - Pontos em aberto no enunciado e decisões adotadas

| Ponto em aberto                      | Decisão                                       | Justificativa                                                                                                     |
|--------------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| A letra Y é consoante?               | Sim. As vogais são A, E, I, O e U.            | Nenhum exemplo do enunciado depende disso. O caso Xyz é recusado pelo tamanho, não pela letra Y.                  |
| Letra acentuada conta como letra?    | Não. É tratada como caractere ilegal.         | O alfabeto adotado é o ASCII, e nenhum exemplo do enunciado usa acento em identificador.                          |
| A vírgula é um símbolo da linguagem? | Sim, com token próprio.                       | Não consta da lista de símbolos, mas aparece no exemplo do professor, na declaração e na chamada de função.       |
| O analisador para no primeiro erro?  | Não. Reporta todos e continua.                | O exemplo do professor tem um erro proposital no meio do arquivo e código válido depois dele.                     |
| migual e MIgual são a mesma palavra? | Não. São duas palavras reservadas diferentes. | O enunciado lista as duas grafias, o que só faz sentido em uma linguagem que diferencia maiúsculas de minúsculas. |

Fonte: Elaborado pelos autores (2026).

## 4 ORGANIZAÇÃO DO PROJETO E DIVISÃO DO TRABALHO

### 4.1 Estrutura do repositório

O repositório tem uma pasta raiz chamada `compiladores-elgol` e, dentro dela, uma subpasta com o mesmo nome, onde fica o projeto propriamente dito. Esse detalhe aparece várias vezes no guia de instalação, porque é preciso entrar duas vezes em `compiladores-elgol` depois de clonar o repositório.

Dentro da subpasta estão a pasta `elgol`, com o analisador léxico (`lexico_elgol.py`), os programas de exemplo e de teste e dois *scripts* de verificação automática (`test_regras.py` e `test_simbolos.py`); a pasta `tr4-c`, com os analisadores léxico e sintático para um subconjunto da linguagem C feitos no TR4; a pasta `docs`, com este artigo e o relatório de conferência das regras léxicas; a pasta `prints-instalacao`, com as capturas de tela da validação no Windows 11 e no Debian 13; e, na raiz da subpasta, o `README.md`, o guia `INSTALACAO.md`, o arquivo `requirements.txt`, que fixa a única dependência (`ply==3.11`), e o exemplo mínimo `instalacao.elg`.

O `README` termina com uma lista de acompanhamento das etapas. No momento da entrega, o TR4 e a Etapa 1 estavam marcados como concluídos, e a Etapa 2 como pendente.

## 4.2 Integrantes e responsabilidades

O grupo tem seis integrantes. O Quadro 3 mostra a contribuição de cada um, de acordo com o histórico de *commits* do repositório e com as indicações do próprio README e do guia de instalação.

Quadro 3 - Integrantes do grupo e suas contribuições

| Integrante                | Contribuição                                                                                                                                                                                                                                                                                |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| André Kroetz Berger       | Guia de instalação (INSTALACAO.md), seções de instalação do README e validação no Windows 11 e no Debian 13, com as capturas da pasta <code>prints-instalacao</code> .                                                                                                                      |
| Arthur Contri Gasperin    | Organização e redação deste artigo.                                                                                                                                                                                                                                                         |
| Guilherme Gabriel Rigotti | Tabela de símbolos impressa pelo analisador e teste <code>test_simbolos.py</code> .                                                                                                                                                                                                         |
| Henrique Marques Kayser   | Criação do repositório, publicação da primeira versão do TR4, do analisador léxico e do artigo da etapa anterior, conferência das regras ( <code>test_regras.py</code> e <code>docs/conferencia_regras.md</code> ), correção do <code>instalacao.elg</code> e integração das contribuições. |
| Lucas Borkert Beck        | Programas com erros intencionais ( <code>teste_erro_1.elg</code> a <code>teste_erro_5.elg</code> ).                                                                                                                                                                                         |
| Rafael Schulz Vaz         | Programas Elgol corretos ( <code>teste_1.elg</code> a <code>teste_4.elg</code> ) e sua execução no analisador.                                                                                                                                                                              |

Fonte: Elaborado pelos autores (2026).

## 5 IMPLEMENTAÇÃO DO ANALISADOR LÉXICO

### 5.1 Validação fora da expressão regular

A regra de identificador de Elgol cabe em uma única expressão regular: uma consoante maiúscula, duas ou mais letras quaisquer e uma letra minúscula no final. O próprio código guarda essa expressão como referência. Usá-la diretamente como padrão do *token*, porém, produz um comportamento errado diante de entradas inválidas. Em `Teste39`, por exemplo, o PLY reconheceria `Teste` como identificador e `39` como número, sem acusar erro algum, embora o enunciado diga que `Teste39` não é um identificador válido. O mesmo aconteceria com `034`, que viraria um erro no zero seguido de um número `34` válido.

A solução adotada inverte a lógica. O padrão de cada regra é propositalmente genérico e captura a palavra inteira, incluindo dígitos e sublinhados: `[A-Za-z_][A-Za-z0-9_]*` para palavras, `[0-9][A-Za-z0-9_]*` para números e `\$[A-Za-z0-9_]*` para nomes de função. Quem decide se o lexema é válido é o código Python dentro da função de regra. As funções `critica_identificador` e `critica_numero` devolvem a lista de regras violadas. Se a

lista vier vazia, o *token* é aceito; caso contrário, o erro é impresso com todos os motivos e o lexema é descartado. Com isso, cada lexema problemático gera exatamente um erro, acompanhado da explicação do que está errado nele.

## 5.2 Prioridade das palavras reservadas

A técnica mais comum para tratar palavras reservadas no PLY é reconhecê-las como identificadores e corrigir o tipo consultando um dicionário. Em Elgol, quase nenhuma palavra reservada se parece com um identificador: algumas começam com minúscula, outras são escritas só com maiúsculas e duas começam com sublinhado. A exceção é `MIgual`, que começa com consoante maiúscula, tem seis letras e termina em minúscula, ou seja, é um identificador perfeitamente válido pela regra. Por isso a função `t_ID` consulta o dicionário de reservadas antes de qualquer validação. Se essa ordem fosse invertida, `MIgual` seria classificado como nome de variável e um operador de comparação desapareceria da linguagem sem nenhum aviso.

A multiplicação exige um cuidado parecido. Como o operador é a letra `x`, ela também está no dicionário de reservadas, com o tipo `VEZES`. Sem isso, o analisador acusaria um identificador inválido de uma letra só toda vez que aparecesse uma multiplicação. Já o comentário, iniciado por `*`, não gera ambiguidade, porque Elgol não usa o asterisco como operador.

## 5.3 Saída, código de retorno e tabela de símbolos

O programa é executado na linha de comando e recebe o caminho do arquivo `.elg`. Sem opções, ele lista cada *token* com linha, coluna, tipo e lexema e, ao final, imprime o total de *tokens* reconhecidos e de erros léxicos. Com a opção `--resumo`, a lista é substituída por uma contagem por tipo de *token*. Quando há pelo menos um erro léxico, o processo termina com código de saída 1, e sem erros termina com 0. Isso permite usar o analisador em *scripts* sem precisar interpretar o texto da saída.

Na versão final do repositório, o modo completo também imprime uma tabela de símbolos depois da lista de *tokens*, recurso incluído em 4 de outubro de 2026. Cada identificador e cada nome de função aparece uma única vez, com o tipo de *token* e as linhas em que ocorre. A Figura 13, na Seção 6, mostra essa tabela para o exemplo do professor. Nas capturas do Windows e nas dos programas de teste, a saída termina na contagem de *tokens* e erros, o que indica que foram feitas com uma versão do código anterior a essa inclusão.

## 6 INSTALAÇÃO E VALIDAÇÃO DO AMBIENTE

### 6.1 Requisitos

Para executar o projeto são necessários o Git, o Python 3.8 ou mais novo e acesso à internet para instalar a única dependência, `ply==3.11`. Os programas Elgol devem ser salvos com codificação UTF-8, que é a codificação usada pelo analisador para abrir os arquivos. O roteiro de instalação aparece em dois lugares: no README, de forma resumida, e no arquivo `INSTALACAO.md`, um guia separado sob responsabilidade de André Kroetz Berger, com instruções para Windows, Debian 13 e Ubuntu e uma tabela de problemas comuns.

Uma escolha atravessa todo o roteiro. A dependência é sempre instalada em um ambiente virtual, criado com o módulo `venv`, e os comandos chamam diretamente o interpretador desse ambiente, em vez de ativá-lo. No Windows, isso dispensa alterar a política de execução de *scripts* do PowerShell. No Linux, evita o erro `externally-managed-environment`, que as versões atuais do Debian e do Ubuntu mostram quando se tenta usar o `pip` no Python do sistema, e torna desnecessários o `sudo pip` e a opção `--break-system-packages`.

### 6.2 Instalação no Windows

No Windows, os comandos são executados no PowerShell. Depois de instalar o Git e o Python, o comando `py -3 --version` confirma a versão do interpretador. Se o inicializador `py` não existir, usa-se `python` no lugar de `py -3`. A sequência de instalação é a seguinte:

```
git clone https://github.com/hmkayser-ops/compiladores-elgol.git
cd compiladores-elgol
cd compiladores-elgol
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip check
```

Os dois comandos `cd` são necessários por causa da subpasta com o mesmo nome do repositório. Ao final, o terminal deve estar na pasta que contém `requirements.txt` e a pasta `elgol`.

### 6.3 Instalação no Debian 13 e no Ubuntu

No Linux, o roteiro usa o Python 3 fornecido pela própria distribuição. Os pré-requisitos são instalados com o `apt`, por uma conta com acesso ao `sudo`. No Debian, se o usuário não tiver essa permissão, o guia orienta entrar como administrador com `su -`, executar os comandos sem `sudo` e voltar ao usuário normal com `exit` antes de clonar o projeto. Os comandos são estes:

```
sudo apt update
sudo apt install git python3 python3-venv python3-pip ca-certificates
git clone https://github.com/hmkayser-ops/compiladores-elgol.git
cd compiladores-elgol/compiladores-elgol
python3 --version
python3 -m venv .venv
./venv/bin/python -m pip install -r requirements.txt
./venv/bin/python -m pip check
```

O guia recomenda usar `python3` sem fixar uma versão, como `python3.13`, para acompanhar a versão padrão de cada distribuição, e criar um ambiente virtual em cada sistema, já que a pasta `.venv` criada no Windows não funciona no Linux. Se a criação do ambiente reclamar do `ensurepip`, é sinal de que o pacote `python3-venv` correspondente à versão do Python não está instalado (Debian, [20--]; Canonical, [20--]).

### 6.4 Conferência da instalação

Para verificar se a instalação funcionou, o README propõe um programa mínimo, salvo como `instalacao.elg` na mesma pasta de `requirements.txt`:

```
inicio .
    DECIMAL Teste .
    Teste = _Z_ .
fim .
```

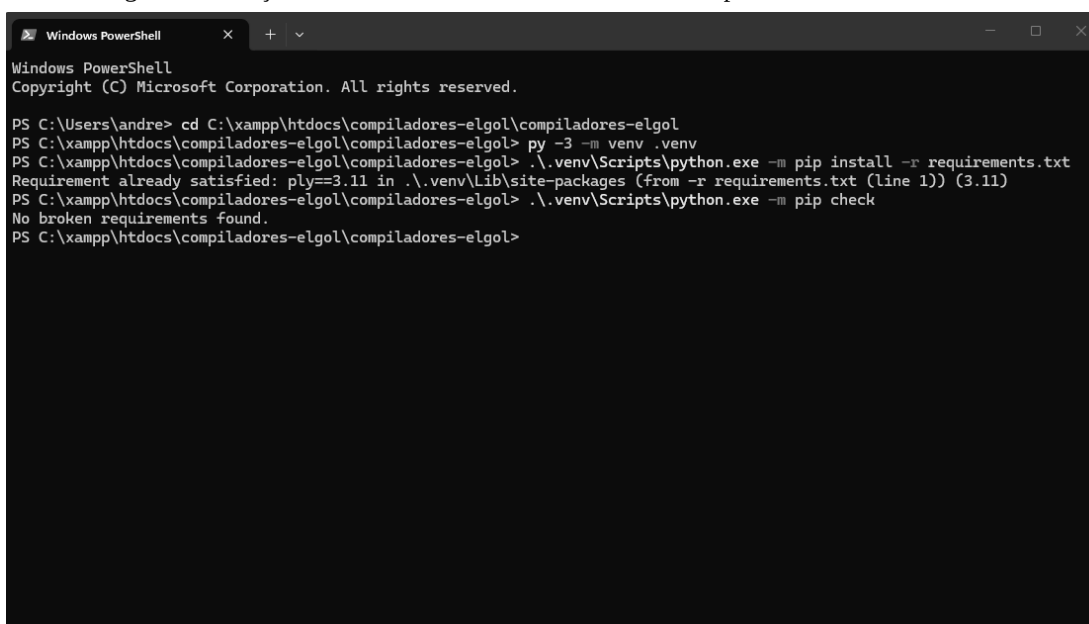
Executado com a opção `--resumo`, esse arquivo deve produzir Tokens reconhecidos: 11, Erros lexicos: 0 e código de saída 0, consultado com `$LASTEXITCODE` no PowerShell ou com `echo $?` no Bash. Em seguida, o roteiro executa os dois exemplos do repositório. O arquivo `exemplo_professor.elg` contém de propósito o identificador inválido `Vim` e deve gerar 75 *tokens* e um erro. O arquivo `casos_borda.elg` reúne os casos citados no enunciado e tem 16 erros intencionais. Os dois terminam com código 1, e esses erros não indicam falha na instalação.

## 6.5 Validação no Windows 11

A validação no Windows foi feita por André Kroetz Berger. Segundo o README, em 3 de outubro de 2026 a instalação em um ambiente virtual novo foi verificada com o Python 3.13.12 e o PLY 3.11. O exemplo mínimo gerou 11 *tokens*, nenhum erro e código de saída 0, o exemplo do professor gerou um erro e o arquivo de casos de borda gerou 16 erros, os dois com código 1, e os comandos do TR4 também foram executados. As Figuras 1 a 7 reproduzem as capturas de tela guardadas na pasta `prints-instalacao/Windows11`.

A Figura 1 mostra a criação do ambiente virtual e a instalação da dependência. Como o ambiente já existia na máquina, o `pip` informa que o PLY 3.11 já estava instalado, e o `pip check` não encontra dependências quebradas.

Figura 1 - Criação do ambiente virtual e conferência da dependência no Windows 11



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\andre> cd C:\xampp\htdocs\compiladores-elgol\compiladores-elgol
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> py -3 -m venv .venv
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> .venv\Scripts\python.exe -m pip install -r requirements.txt
Requirement already satisfied: ply==3.11 in .venv\Lib\site-packages (from -r requirements.txt (line 1)) (3.11)
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> .venv\Scripts\python.exe -m pip check
No broken requirements found.
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol>
  
```

Fonte: Elaborada pelos autores (2026), arquivo Windows11-01.png do repositório.

A Figura 2 merece uma explicação. A execução do exemplo mínimo terminou com 16 *tokens*, um erro léxico no lexema `ply` e código de saída 1, quando o esperado eram 11 *tokens* e nenhum erro. A causa não está no analisador. A versão do `instalacao.elg` enviada junto com as capturas tinha, antes do programa, a linha `ply==3.11`, que é o conteúdo do `requirements.txt`. O analisador fez exatamente o que deveria ao recusar `ply`, que começa com minúscula e tem só três caracteres, e ao reconhecer o resto da linha como dois sinais de atribuição, dois números e um ponto. O arquivo foi corrigido depois, em um *commit* específico, e a versão atual produz 11 *tokens* e nenhum erro.

Figura 2 - Execução do instalacao.elg no Windows 11, ainda com a linha `ply==3.11` no início do arquivo

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\andre> cd C:\xampp\htdocs\compiladores-elgol\compiladores-elgol
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> py -3 -m venv .venv
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> .\venv\Scripts\python.exe -m pip install -r requirements.txt
Requirement already satisfied: ply==3.11 in .\venv\Lib\site-packages (from -r requirements.txt (line 1)) (3.11)
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> .\venv\Scripts\python.exe -m pip check
No broken requirements found.
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> .\venv\Scripts\python.exe elgol\lexico_elgol.py instalacao.elg --resumo
[ERRO LEXICO] linha 1, coluna 1: 'ply'
                nao e um identificador valido:
                deve comecar com letra maiuscula
                precisa de no minimo 4 caracteres (tem 3)

-----
Tokens reconhecidos: 16
Erros lexicos: 1
  ATRIB          3
  DECIMAL        1
  FIM             1
  ID              2
  INICIO          1
  NUMERO          2
  PONTO           5
  ZERO            1
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> $LASTEXITCODE
1
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol>

```

Fonte: Elaborada pelos autores (2026), arquivo Windows11-02.png do repositório.

As Figuras 3, 4 e 5 mostram a execução completa do exemplo do professor. A Figura 3 traz o início da lista de *tokens*, com a declaração da função `$Soma` e de seus parâmetros. A Figura 4 mostra o erro na linha 14, coluna 11, onde o identificador `Vim` é recusado por ter apenas três caracteres, e a análise seguindo normalmente depois dele. A Figura 5 traz o final da lista, com 75 *tokens* reconhecidos, um erro e código de saída 1.

Figura 3 - Início da análise do exemplo do professor no Windows 11

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> .\venv\Scripts\python.exe elgol\lexico_elgol.py elgol\exemplo_professor.elg

```

| LINHA | COL | TOKEN     | LEXEMA    |
|-------|-----|-----------|-----------|
| 5     | 1   | DECIMAL   | 'DECIMAL' |
| 5     | 9   | FUNCAO    | '\$Soma'  |
| 5     | 15  | ABRE_PAR  | '('       |
| 5     | 16  | DECIMAL   | 'DECIMAL' |
| 5     | 24  | ID        | 'Numm'    |
| 5     | 28  | VIRGULA   | '.'       |
| 5     | 30  | DECIMAL   | 'DECIMAL' |
| 5     | 38  | ID        | 'Dois'    |
| 5     | 42  | FECHA_PAR | ')'       |
| 5     | 44  | PONTO     | '.'       |
| 6     | 1   | INICIO    | 'inicio'  |
| 6     | 8   | PONTO     | '.'       |
| 7     | 5   | ELGIO     | 'elgio'   |
| 7     | 11  | ATRIB     | '='       |
| 7     | 13  | ID        | 'Numm'    |
| 7     | 18  | MAIS      | '+'       |
| 7     | 20  | ID        | 'Doiss'   |
| 7     | 26  | PONTO     | '.'       |
| 8     | 1   | FIM       | 'fim'     |
| 8     | 5   | PONTO     | '.'       |
| 11    | 1   | INICIO    | 'inicio'  |
| 11    | 8   | PONTO     | '.'       |
| 12    | 3   | DECIMAL   | 'DECIMAL' |
| 12    | 11  | ID        | 'Lixo'    |
| 12    | 16  | PONTO     | '.'       |
| 13    | 3   | DECIMAL   | 'DECIMAL' |
| 13    | 11  | ID        | 'Teste'   |

Fonte: Elaborada pelos autores (2026), arquivo Windows11-03.png do repositório.

Figura 4 - Erro léxico no identificador Vim, linha 14, coluna 11

```

Windows PowerShell
13 17 PONTO '.'
14 3 DECIMAL 'DECIMAL'
[ERRO LEXICO] linha 14, coluna 11: 'Vim'
nao e um identificador valido:
precisa de no minimo 4 caracteres (tem 3)
14 15 PONTO '.'
16 3 ID 'Lixo'
16 8 ATRIB '='
16 10 NUMERO '34'
16 13 PONTO '.'
17 3 NEG '_NEG_'
17 9 ID 'Lixo'
17 14 PONTO '.'
19 3 SE 'se'
19 6 ID 'Lixo'
19 11 MAIOR 'maior'
19 17 ZERO '_Z_'
19 21 PONTO '.'
20 3 ENTAO 'entao'
20 8 PONTO '.'
21 3 INICIO 'inicio'
21 9 PONTO '.'
22 9 ID 'Lixo'
22 14 ATRIB '='
22 16 ZERO '_Z_'
22 19 PONTO '.'
23 9 ID 'Teste'
23 15 ATRIB '='
23 17 NUMERO '3'
23 19 EXP 'EXP'

```

Fonte: Elaborada pelos autores (2026), arquivo Windows11-04.png do repositório.

Figura 5 - Final da análise do exemplo do professor, com 75 tokens e um erro

```

Windows PowerShell
23 15 ATRIB '='
23 17 NUMERO '3'
23 19 EXP 'EXP'
23 23 NUMERO '4'
23 25 PONTO '.'
24 9 ID 'Lixo'
24 14 ATRIB '='
24 16 NUMERO '13'
24 19 RESTO 'RESTO'
24 25 NUMERO '4'
24 27 PONTO '.'
25 9 ID 'Resultado'
25 19 ATRIB '='
25 21 FUNCAO '$Soma'
25 27 ABRE_PAR '('
25 28 ID 'Lixo'
25 32 VIRGULA ','
25 34 ID 'Teste'
25 39 FECHA_PAR ')'
25 41 PONTO '.'
26 3 FIM 'fim'
26 6 PONTO '.'
27 1 FIM 'fim'
27 5 PONTO '.'

Tokens reconhecidos: 75
Erros lexicos: 1
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> $LASTEXITCODE
1
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol>

```

Fonte: Elaborada pelos autores (2026), arquivo Windows11-05.png do repositório.

As Figuras 6 e 7 mostram a execução de `casos_borda.elg` com a opção `--resumo`. Cada caso inválido aparece como um único erro, com todas as regras que violou: `teste` não começa com maiúscula, `Teste39` contém dígitos e não termina com minúscula, `Tes_Te` contém sublinhado, `Xyz`, `Bh` e `Lxt` são curtos demais, `LetrA` termina com maiúscula e `Ateras` começa com vogal. No final aparecem os nomes de função inválidos `$Te34` e `$Te` e o caractere ilegal `@`, com o total de 111 *tokens* e 16 erros.

Figura 6 - Primeiros erros apontados em casos\_borda.elg

```

Windows PowerShell
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> .\venv\Scripts\python.exe elgol\lexico_elgol.py elgol\casos_borda.elg --resumo

[ERRO LEXICO] linha 9, coluna 3: 'teste'
              nao e um identificador valido:
              deve comecar com letra maiuscula
[ERRO LEXICO] linha 10, coluna 3: 'teste2'
              nao e um identificador valido:
              so pode conter letras (encontrado: '2')
              deve comecar com letra maiuscula
              deve terminar com letra minuscula
[ERRO LEXICO] linha 11, coluna 3: 'Teste39'
              nao e um identificador valido:
              so pode conter letras (encontrado: '3' '9')
              deve terminar com letra minuscula
[ERRO LEXICO] linha 12, coluna 3: 'Tes_Le'
              nao e um identificador valido:
              so pode conter letras (encontrado: '_')
[ERRO LEXICO] linha 13, coluna 3: 'Xyz'
              nao e um identificador valido:
              precisa de no minimo 4 caracteres (tem 3)
[ERRO LEXICO] linha 14, coluna 3: 'Bh'
              nao e um identificador valido:
              precisa de no minimo 4 caracteres (tem 2)
[ERRO LEXICO] linha 15, coluna 3: 'Lxt'
              nao e um identificador valido:
              precisa de no minimo 4 caracteres (tem 3)
[ERRO LEXICO] linha 16, coluna 3: 'LetrA'
              nao e um identificador valido:
              deve terminar com letra minuscula
[ERRO LEXICO] linha 17, coluna 3: 'Ateras'

```

Fonte: Elaborada pelos autores (2026), arquivo Windows11-06.png do repositório.

Figura 7 - Últimos erros e contagem por tipo de token em casos\_borda.elg

```

Windows PowerShell
[ERRO LEXICO] linha 28, coluna 11: '$Te34'
              nome de funcao invalido:
              so pode conter letras (encontrado: '3' '4')
              deve terminar com letra minuscula
[ERRO LEXICO] linha 29, coluna 11: '$Te'
              nome de funcao invalido:
              precisa de no minimo 4 caracteres (tem 2)
[ERRO LEXICO] linha 41, coluna 16: caractere ilegal '@'

-----
Tokens reconhecidos: 111
Erros lexicos: 16
ABRE_PAR      4
ATRIB         21
DECIMAL       3
DIVIDE        1
EXP           1
FECHA_PAR     4
FIM           1
FUNCAO        1
ID            20
INICIO        1
MAIGUAL       1
MENOS         1
MIGUAL        1
NUMERO        19
PONTO         28
RESTO         1
SE            2
VEZES         1
PS C:\xampp\htdocs\compiladores-elgol\compiladores-elgol> $LASTEXITCODE

```

Fonte: Elaborada pelos autores (2026), arquivo Windows11-07.png do repositório.

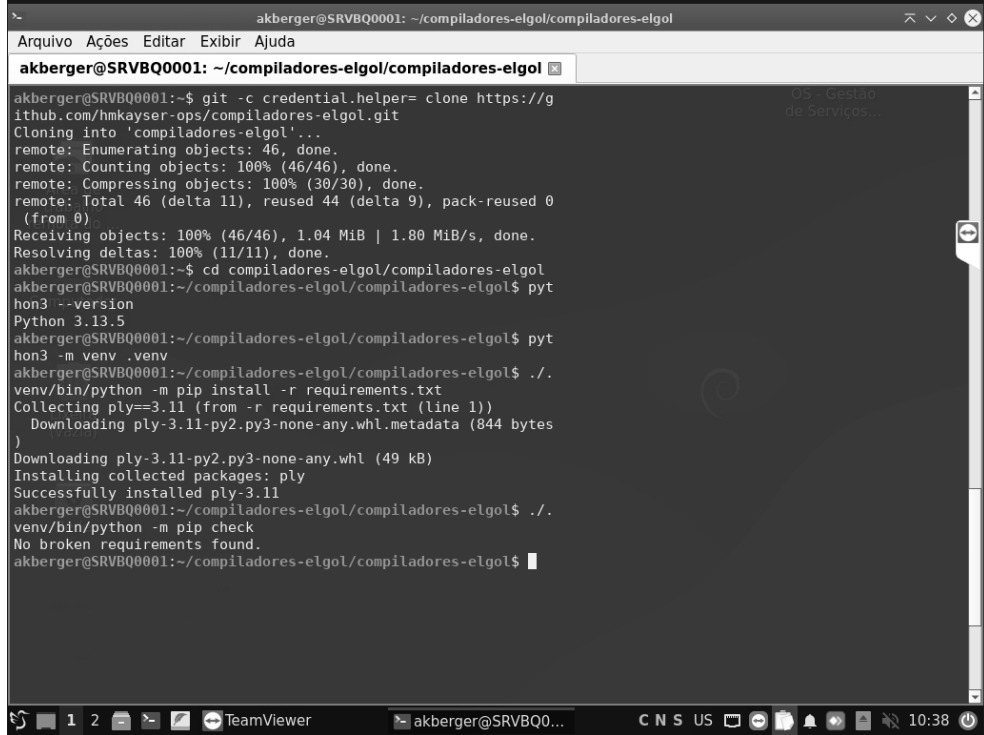
## 6.6 Validação no Debian 13

Em 5 de outubro de 2026, o mesmo roteiro foi executado no Debian 13 com o Python 3.13 e o PLY 3.11, clonando a ramificação principal. A instalação terminou sem erros e o pip check não apontou problemas. As Figuras 8 a 13 reproduzem as imagens da pasta prints-instalacao/Linux-Debian13.

Na Figura 8, o repositório é clonado, o comando `python3 --version` mostra o Python 3.13.5, o ambiente virtual é criado e o PLY 3.11 é baixado e instalado. A Figura 9 mostra, no gerenciador de arquivos, o conteúdo da subpasta do projeto depois do clone: as

pastas docs, elgol e tr4-c e os arquivos instalacao.elg, README.md e requirements.txt.

Figura 8 - Clone do repositório, criação do ambiente virtual e instalação do PLY no Debian 13



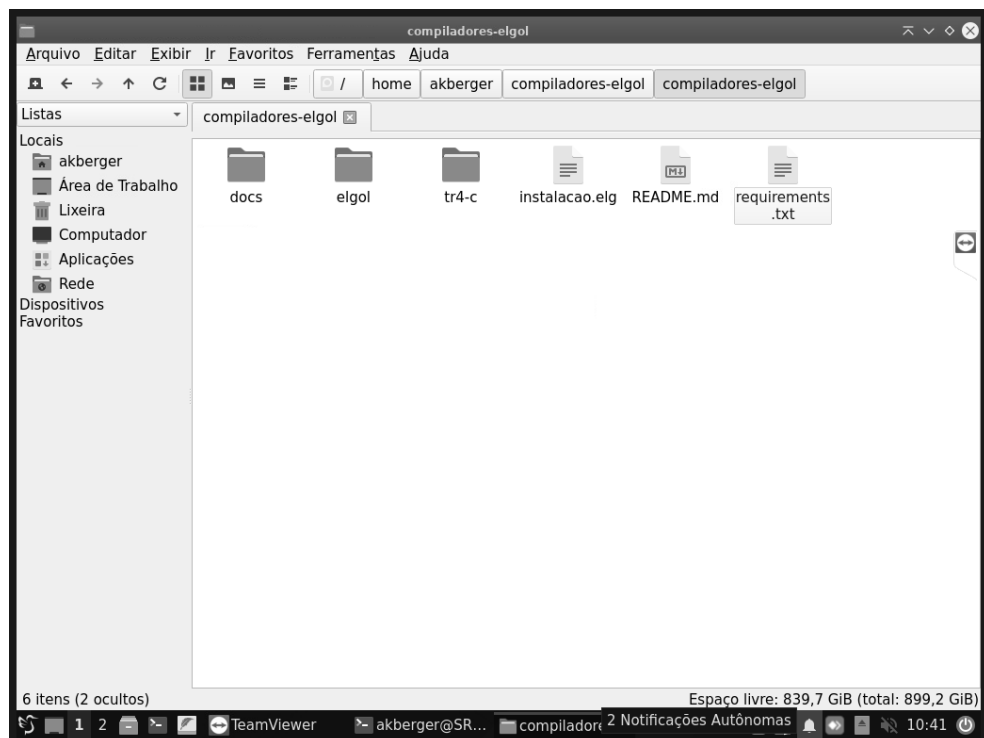
```

akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol
Arquivo  Ações  Editar  Exibir  Ajuda

akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol
akberger@SRVBQ0001:~$ git -c credential.helper= clone https://g
ithub.com/hmkayser-ops/compiladores-elgol.git
Cloning into 'compiladores-elgol'...
remote: Enumerating objects: 46, done.
remote: Counting objects: 100% (46/46), done.
remote: Compressing objects: 100% (30/30), done.
remote: Total 46 (delta 11), reused 44 (delta 9), pack-reused 0
(from 0)
Receiving objects: 100% (46/46), 1.04 MiB | 1.80 MiB/s, done.
Resolving deltas: 100% (11/11), done.
akberger@SRVBQ0001:~$ cd compiladores-elgol/compiladores-elgol
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$ pyt
hon3 --version
Python 3.13.5
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$ pyt
hon3 -m venv .venv
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$ ./
venv/bin/python -m pip install -r requirements.txt
Collecting ply==3.11 (from -r requirements.txt (line 1))
  Downloading ply-3.11-py2.py3-none-any.whl.metadata (844 bytes
)
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
Installing collected packages: ply
Successfully installed ply-3.11
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$ ./
venv/bin/python -m pip check
No broken requirements found.
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$
  
```

Fonte: Elaborada pelos autores (2026), arquivo DEBIAN-01.png do repositório.

Figura 9 - Conteúdo da subpasta do projeto no gerenciador de arquivos do Debian 13



Fonte: Elaborada pelos autores (2026), arquivo DEBIAN-02.png do repositório.

A Figura 10 repete, no Linux, a situação da Figura 2. Com a linha `ply==3.11` ainda presente no início do `instalacao.elg`, a saída mostra 16 *tokens*, um erro e código de saída 1. O comportamento é idêntico nos dois sistemas, o que reforça que a causa está no conteúdo do arquivo e não no ambiente.

Figura 10 - Execução do `instalacao.elg` no Debian 13, ainda com a linha `ply==3.11` no início do arquivo

```

akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol
Arquivo Ações Editar Exibir Ajuda
akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol
hon3 --version
Python 3.13.5
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$ pyt
hon3 -m venv .venv
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$ ./
venv/bin/python -m pip install -r requirements.txt
Collecting ply==3.11 (from -r requirements.txt (line 1))
  Downloading ply-3.11-py2.py3-none-any.whl.metadata (844 bytes)
    Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
Installing collected packages: ply
Successfully installed ply-3.11
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$ ./
venv/bin/python -m pip check
No broken requirements found.
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$ ./venv/bin/python elgol/lexico_elgol.py instalaca
o.elg --resumo
echo $?
[ERRO LEXICO] linha 1, coluna 1: 'ply'
              nao e um identificador valido:
              deve comecar com letra maiuscula
              precisa de no minimo 4 caracteres (tem 3)
-----
Tokens reconhecidos: 16
Erros lexicos: 1
ATRIB          3
DECIMAL        1
FIM            1
ID             2
INICIO         1
NUMERO         2
PONTO          5
ZERO           1
1
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$

```

Fonte: Elaborada pelos autores (2026), arquivo DEBIAN-03.png do repositório.

As Figuras 11, 12 e 13 mostram a execução do exemplo do professor no Debian. A lista de *tokens* coincide com a obtida no Windows, inclusive no erro do Vim. A Figura 13 traz, além do total de 75 *tokens* e um erro, a tabela de símbolos com sete entradas: a função `$Soma` e os identificadores `Numm`, `Dois`, `Doiss`, `Lixo`, `Teste` e `Resultado`, cada um com as linhas em que aparece. O README registra que o Ubuntu não chegou a ser testado.

Figura 11 - Início da análise do exemplo do professor no Debian 13

```

akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol
Arquivo  Ações  Editar  Exibir  Ajuda
akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol$ ./venv/bin/python elgol/lexico_elgol.py elgol/exe
mplo_professor.elg
echo $?
LINHA COLAR TOKEN LEXEMA
-----
5 1 Área de trabalho DECIMAL 'DECIMAL'
5 5 Lixo a FUNCAO '$Soma'
5 15 Cor15utador ABRE_PAR '('
5 16 DECIMAL 'DECIMAL'
5 24 Apl2ações ID 'Numm'
5 28 Rec VIRGULA ','
5 30 spositi DECIMAL 'DECIMAL'
5 38 oritos ID 'Dois'
5 42 FECHA_PAR ')'
5 44 PONTO '.'
6 1 INICIO 'inicio'
6 8 PONTO '.'
7 5 ELGIO 'elgio'
7 11 ATRIB '='
7 13 ID 'Numm'
7 18 MAIS '+'
7 20 ID 'Doiss'
7 26 PONTO '.'
8 1 FIM 'fim'
8 5 PONTO '.'
11 1 INICIO 'inicio'
11 8 PONTO '.'
12 3 DECIMAL 'DECIMAL'
12 11 ID 'Lixo'
12 16 PONTO '.'
13 3 DECIMAL 'DECIMAL'
13 11 ID 'Teste'
13 17 PONTO '.'
14 3 DECIMAL 'DECIMAL'
[ERRO LEXICO] linha 14, coluna 11: 'Vim'
6 itens (2 ocultos)

```

Fonte: Elaborada pelos autores (2026), arquivo DEBIAN-04.png do repositório.

Figura 12 - Erro no identificador Vim e continuação da análise no Debian 13

```

akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol
Arquivo  Ações  Editar  Exibir  Ajuda
akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol$ ./venv/bin/python elgol/lexico_elgol.py elgol/exe
mplo_professor.elg
echo $?
LINHA COLAR TOKEN LEXEMA
-----
14 3 DECIMAL 'DECIMAL'
[ERRO LEXICO] linha 14, coluna 11: 'Vim'
nao e um identificador valido:
precisa de no minimo 4 caracteres (tem 3)
14 15 Área de Trabalho PONTO '.'
16 5 Lixo a ID docs 'Lixo'
16 10 Cor15utador ATRIB '='
16 13 Apl3ações NUMERO 34
17 28 Rec PONTO '.'
17 30 spositi NEG 'NEG'
19 3 oritos ID 'Lixo'
19 6 SE 'se'
19 11 ID 'Lixo'
19 17 MAIOR 'maior'
19 21 ZERO 'Z'
20 3 PONTO '.'
20 8 ENTAO 'entao'
21 3 PONTO '.'
21 9 INICIO 'inicio'
22 9 PONTO '.'
22 14 ID 'Lixo'
22 16 ATRIB '='
22 19 ZERO 'Z'
23 9 PONTO '.'
23 15 ID 'Teste'
23 17 ATRIB '='
23 19 NUMERO 3
23 23 EXP 'EXP'
23 25 NUMERO 4
24 9 PONTO '.'
24 14 ID 'Lixo'
24 16 ATRIB '='
24 19 NUMERO 13
24 24 RESTO 'RESTO'
6 itens (2 ocultos)

```

Fonte: Elaborada pelos autores (2026), arquivo DEBIAN-05.png do repositório.

Figura 13 - Final da análise e tabela de símbolos do exemplo do professor no Debian 13

```

akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol
Arquivo  Ações  Editar  Exibir  Ajuda

akberger@SRVBQ0001: ~/compiladores-elgol/compiladores-elgol
24 as 16 NUMERO 13
24 as 19 RESTO 'RESTO'
24 as 25 NUMERO 4
24 as 27ger PONT0 ' '
25 9 ID 'Resultado'
25 19 ATRIB ocs '=;'
25 21 Lix21 FUNCAO '$Soma'
25 27 Autador ABRE_PAR '('
25 28 ID 'Lixo'
25 32ções VIRGULA ','
25 34 ID 'Teste'
25 39 FECHA_PAR ')'
25 41 PONT0 ' '
26 3 FIM 'fim'
26 6 PONT0 ' '
27 1 FIM 'fim'
27 5 PONT0 ' '

Tokens reconhecidos: 75
Erros lexicos: 1

TABELA DE SIMBOLOS
# LEXEMA TOKEN LINHAS
-----
1 $Soma FUNCAO 5, 25
2 Numm ID 5, 7
3 Dois ID 5
4 Doiss ID 7
5 Lixo ID 12, 16, 17, 19, 22, 24, 25
6 Teste ID 13, 23, 25
7 Resultado ID 25

Simbolos: 7
1
akberger@SRVBQ0001:~/compiladores-elgol/compiladores-elgol$

```

Fonte: Elaborada pelos autores (2026), arquivo DEBIAN-6.png do repositório.

A Tabela 1 resume o que as capturas registram nos dois sistemas e o resultado obtido com a versão atual do `instalacao.elg`. Na Figura 7, a captura termina antes de o código de saída aparecer, e a execução de `casos_borda.elg` não foi registrada nas imagens do Debian.

Tabela 1 - Resultados registrados na validação da instalação

| Arquivo                            | Origem do resultado     | Tokens | Erros | Código de saída |
|------------------------------------|-------------------------|--------|-------|-----------------|
| <code>instalacao.elg</code>        | Captura no Windows 11   | 16     | 1     | 1               |
| <code>instalacao.elg</code>        | Captura no Debian 13    | 16     | 1     | 1               |
| <code>instalacao.elg</code>        | Versão atual do arquivo | 11     | 0     | 0               |
| <code>exemplo_professor.elg</code> | Captura no Windows 11   | 75     | 1     | 1               |
| <code>exemplo_professor.elg</code> | Captura no Debian 13    | 75     | 1     | 1               |
| <code>casos_borda.elg</code>       | Captura no Windows 11   | 111    | 16    | não visível     |

Fonte: Elaborada pelos autores (2026).

## 6.7 Problemas comuns

O guia de instalação termina com uma tabela de problemas frequentes e de como resolvê-los, reproduzida no Quadro 4.

Quadro 4 - Problemas comuns na instalação e como resolvê-los

| Mensagem ou situação                        | Como resolver                                                                             |
|---------------------------------------------|-------------------------------------------------------------------------------------------|
| git, py ou python não encontrado no Windows | Conferir a instalação e a configuração do PATH e abrir um novo terminal.                  |
| requirements.txt não encontrado             | Entrar na subpasta compiladores-elgol dentro do clone.                                    |
| ensurepip indisponível no Linux             | Instalar o python3-venv do Python padrão da distribuição.                                 |
| externally-managed-environment              | Usar <code>./venv/bin/python -m pip</code> , dentro do ambiente virtual.                  |
| ModuleNotFoundError: No module named 'ply'  | Instalar o requirements.txt com o mesmo Python da .venv usado para executar o analisador. |
| Falha ao baixar o PLY                       | Conferir a conexão com a internet e as configurações de proxy da rede.                    |
| Arquivo .elg não encontrado                 | Conferir o caminho e a extensão do arquivo.                                               |

Fonte: Elaborado pelos autores (2026).

## 7 PROGRAMAS DE TESTE EM ELGOL

### 7.1 Objetivo

O exemplo do professor e os casos de borda servem para conferir as regras uma a uma, mas não mostram o analisador lidando com programas completos e corretos. Essa parte do trabalho, feita por Rafael Schulz Vaz, consistiu em escrever programas Elgol válidos que usassem os recursos permitidos pelo enunciado, como os laços enquanto e para, o desvio com senao, a declaração e a chamada de funções, os comentários e as palavras `_Z_` e `_NEG_`, e executá-los no analisador. O critério de sucesso é simples: um programa correto precisa passar sem nenhum erro léxico e com cada lexema classificado no tipo certo.

Foram escritos quatro programas, salvos em `elgol/teste_1.elg` a `elgol/teste_4.elg`. Todos seguem a mesma estrutura: um comentário de cabeçalho que descreve o tema e os elementos usados, a declaração de uma função com `DECIMAL`, cujo corpo fica entre `inicio` e `fim` e atribui o valor calculado à palavra reservada `elgio`, e em seguida o bloco principal do programa. O código completo, com a numeração das linhas, está no Apêndice A. As Figuras 14 a 17 mostram o final da lista de *tokens* de cada programa e os totais. As execuções foram feitas no Windows com o comando `python lexico_elgol.py`, a partir da pasta `elgol`.

## 7.2 Programa teste\_1: aprovação de aluno

O primeiro programa calcula a média de duas notas por meio da função `$Media`, que recebe dois parâmetros separados por vírgula, e compara o resultado com uma meta usando o operador `MIgual`. Conforme o caso, a variável `Resultado` recebe 1 ou `_Z_`, dentro de um desvio com `se`, `entao` e `senao`. O programa exercita a vírgula, a divisão, a soma e a palavra `_Z_` no lugar do zero. A análise reconheceu 91 *tokens*, sem erros. Na Figura 14 aparece o desvio final, com `_Z_` classificado como ZERO na linha 31.

Figura 14 - Final da análise de teste\_1.elg

```
PS C:\Users\Rafael\Desktop\compiladores-elgol\compiladores-elgol\compiladores-elgol\elgol> python lexico_elgol.py teste_1.elg
24 25 ID 'Meta'
24 30 PONTO '.'
25 5 ENTAO 'entao'
25 11 PONTO '.'
26 5 INICIO 'inicio'
26 12 PONTO '.'
27 9 ID 'Resultado'
27 19 ATRIB '='
27 21 NUMERO 1
27 23 PONTO '.'
28 5 FIM 'fim'
28 9 PONTO '.'
29 5 SENAO 'senao'
29 11 PONTO '.'
30 5 INICIO 'inicio'
30 12 PONTO '.'
31 9 ID 'Resultado'
31 19 ATRIB '='
31 21 ZERO '_Z_'
31 25 PONTO '.'
32 5 FIM 'fim'
32 9 PONTO '.'
33 1 FIM 'fim'
33 5 PONTO '.'
-----
Tokens reconhecidos: 91
Erros lexicos: 0
```

Fonte: Elaborada pelos autores (2026), execução de teste\_1.elg.

## 7.3 Programa teste\_2: desconto em compras

O segundo programa aplica um desconto ao valor de uma compra. A função `$Desconto` calcula um fator com `3 EXP 2` e atribui a `elgio` a expressão `(Valor x Fator) / 10`, o que exercita a potência, a multiplicação com a letra `x`, os parênteses em expressão e um comentário no meio da linha, depois do ponto final. O bloco principal usa `RESTO`, os comparadores `menor` e `maior` e dois desvios `se` aninhados. Foram 96 *tokens*, sem erros. A Figura 15 mostra a chamada `$Desconto (ValorCompra)` na linha 27, separada corretamente em `FUNCAO`, `ABRE_PAR`, `ID` e `FECHA_PAR`.

Figura 15 - Final da análise de teste\_2.elg

```

PS C:\Users\Rafael\Desktop\compiladores-elgol\compiladores-elgol\compiladores-elgol> python lexico_elgol.py teste_2.elg
24 24 MAIOR 'maior'
24 30 ID 'Limite'
24 37 PONTO '.'
25 9 ENTAO 'entao'
25 15 PONTO '.'
26 9 INICIO 'inicio'
26 16 PONTO '.'
27 13 ID 'ValorCompra'
27 25 ATRIB '='
27 27 FUNCAO '$Desconto'
27 37 ABRE_PAR '('
27 38 ID 'ValorCompra'
27 49 FECHA_PAR ')'
27 51 PONTO '.'
28 9 FIM 'fim'
28 13 PONTO '.'
29 5 FIM 'fim'
29 9 PONTO '.'
30 5 SENAO 'senao'
30 11 PONTO '.'
31 5 INICIO 'inicio'
31 12 PONTO '.'
32 9 ID 'ValorCompra'
32 21 ATRIB '='
32 23 ID 'ValorCompra'
32 35 PONTO '.'
33 5 FIM 'fim'
33 9 PONTO '.'
34 1 FIM 'fim'
34 5 PONTO '.'
-----
Tokens reconhecidos: 96
Erros lexicos: 0

```

Fonte: Elaborada pelos autores (2026), execução de teste\_2.elg.

#### 7.4 Programa teste\_3: fatorial e contagem

O terceiro programa é o que trata dos laços. A função `$Fatorial` multiplica um acumulador dentro de um enquanto, decrementando o valor a cada passo, e o bloco principal usa um laço para com o operador `migual`, contendo um se com maior. O programa também usa a subtração e a soma. Foram 97 *tokens*, sem erros, e a Figura 16 mostra o laço para na linha 27, com `migual` classificado como `MIGUAL`.

Figura 16 - Final da análise de teste\_3.elg

```

PS C:\Users\Rafael\Desktop\compiladores-elgol\compiladores-elgol\compiladores-elgol> python lexico_elgol.py teste_3.elg
25 13 NUMERO 1
25 15 PONTO '.'
27 5 PARA 'para'
27 10 ID 'Passo'
27 16 MIGUAL 'migual'
27 23 NUMERO 10
27 26 PONTO '.'
28 5 INICIO 'inicio'
28 12 PONTO '.'
29 9 SE 'se'
29 12 ID 'Passo'
29 18 MAIOR 'maior'
29 24 NUMERO 5
29 26 PONTO '.'
30 9 ENTAO 'entao'
30 15 PONTO '.'
31 9 INICIO 'inicio'
31 16 PONTO '.'
32 13 ID 'Resultado'
32 23 ATRIB '='
32 25 ID 'Resultado'
32 35 MAIS '+'
32 37 NUMERO 1
32 39 PONTO '.'
33 9 FIM 'fim'
33 13 PONTO '.'
34 5 FIM 'fim'
34 9 PONTO '.'
35 1 FIM 'fim'
35 5 PONTO '.'
-----
Tokens reconhecidos: 97
Erros lexicos: 0

```

Fonte: Elaborada pelos autores (2026), execução de teste\_3.elg.

Na revisão da lista completa de *tokens*, a linha 10 trazia a grafia `Migual` no lugar de `migual`. Como Elgol diferencia maiúsculas de minúsculas, o analisador classificou a palavra como identificador, e não como operador. Era um erro de digitação, corrigido no repositório, e a contagem de 97 *tokens* sem erros não mudou.

## 7.5 Programa teste\_4: validador de conta

O quarto programa é o que usa as palavras especiais de valor. A função `$VerificaConta` testa se o saldo é diferente de `_Z_` e atribui a `elgio` o valor 1 ou `_Z_`. No bloco principal, a variável `Devedor` recebe `_NEG_ SaldoAtual`, que é a forma de Elgol representar um valor negativo, já que o enunciado não permite escrever o sinal de menos antes de um número. Foram 72 *tokens*, sem erros. A Figura 17 mostra `_NEG_` classificado como `NEG` na linha 28 e a chamada da função na linha 30.

Figura 17 - Final da análise de teste\_4.elg

```

PS C:\Users\Rafael\Desktop\compiladores-elgol\compiladores-elgol\compiladores-elgol> python lexico_elgol.py teste_4.elg
20 5 PONTO '.'
22 1 INICIO 'inicio'
22 8 PONTO '.'
23 5 DECIMAL 'DECIMAL'
23 13 ID 'SaldoAtual'
23 24 PONTO '.'
24 5 DECIMAL 'DECIMAL'
24 13 ID 'Devedor'
24 21 PONTO '.'
25 5 DECIMAL 'DECIMAL'
25 13 ID 'Retorno'
25 21 PONTO '.'
27 5 ID 'SaldoAtual'
27 16 ATRIB '='
27 18 NUMERO 50
27 21 PONTO '.'
28 5 ID 'Devedor'
28 13 ATRIB '='
28 15 NEG '_NEG_'
28 21 ID 'SaldoAtual'
28 32 PONTO '.'
30 5 ID 'Retorno'
30 13 ATRIB '='
30 15 FUNCAO '$VerificaConta'
30 30 ABRE_PAR '('
30 31 ID 'Devedor'
30 38 FECHA_PAR ')'
30 40 PONTO '.'
31 1 FIM 'fim'
31 5 PONTO '.'
-----
Tokens reconhecidos: 72
Erros lexicos: 0

```

Fonte: Elaborada pelos autores (2026), execução de teste\_4.elg.

## 7.6 Resultados e cobertura

A Tabela 2 resume as quatro execuções. Os totais das capturas foram conferidos executando novamente os mesmos arquivos com o PLY 3.11, e coincidem.

Tabela 2 - Resultado da análise léxica dos programas corretos

| Programa     | Linhas     | Tokens     | Erros léxicos | Código de saída |
|--------------|------------|------------|---------------|-----------------|
| teste_1.elg  | 33         | 91         | 0             | 0               |
| teste_2.elg  | 34         | 96         | 0             | 0               |
| teste_3.elg  | 35         | 97         | 0             | 0               |
| teste_4.elg  | 31         | 72         | 0             | 0               |
| <b>Total</b> | <b>133</b> | <b>356</b> | <b>0</b>      |                 |

Fonte: Elaborada pelos autores (2026).

O Quadro 5 mostra quais recursos aparecem em cada programa, com base na contagem por tipo de *token* gerada pela opção - -resumo. Juntos, os quatro programas cobrem todos os recursos pedidos: enquanto e para no teste\_3, senao nos testes 1, 2 e 4, funções e comentários nos quatro, \_Z\_ nos testes 1 e 4 e \_NEG\_ no teste\_4. Todos os operadores aparecem pelo menos uma vez. Das 19 palavras reservadas, 18 aparecem como tais. A única ausente é

igual, citada no comentário de cabeçalho do teste\_3, mas que não aparece no código de nenhum dos programas.

Quadro 5 - Recursos da linguagem usados em cada programa de teste

| Recurso                        | teste_1 | teste_2      | teste_3       | teste_4   |
|--------------------------------|---------|--------------|---------------|-----------|
| Laço enquanto                  |         |              | X             |           |
| Laço para                      |         |              | X             |           |
| Desvio se e entao              | X       | X            | X             | X         |
| Desvio com senao               | X       | X            |               | X         |
| Declaração e chamada de função | X       | X            | X             | X         |
| Comentário                     | X       | X            | X             | X         |
| _Z_                            | X       |              |               | X         |
| _NEG_                          |         |              |               | X         |
| EXP e RESTO                    |         | X            |               |           |
| Multiplicação com x            |         | X            | X             |           |
| Comparadores usados            | MIgual  | menor, maior | migual, maior | diferente |

Fonte: Elaborado pelos autores (2026).

## 7.7 Programas com erros intencionais

Para complementar, Lucas Borkert Beck escreveu cinco programas com erros propositais, salvos em `elgol/teste_erro_1.elg` a `elgol/teste_erro_5.elg`. Cada um parte do mesmo programa de aprovação de aluno do teste\_1 e introduz uma única categoria de erro, indicada em um comentário na linha onde ela ocorre. Esses programas não têm capturas de tela no repositório. A Tabela 3 mostra o resultado obtido ao executá-los no analisador.

Tabela 3 - Resultado da análise léxica dos programas com erros intencionais

| Arquivo          | Categoria do erro                       | Lexema  | Tokens | Erros |
|------------------|-----------------------------------------|---------|--------|-------|
| teste_erro_1.elg | Identificador com menos de 4 caracteres | Not     | 86     | 5     |
| teste_erro_2.elg | Número com zero à esquerda              | 05      | 90     | 1     |
| teste_erro_3.elg | Símbolo fora do alfabeto da linguagem   | @       | 96     | 1     |
| teste_erro_4.elg | Dígito em identificador                 | Media1  | 86     | 5     |
| teste_erro_5.elg | Nome de função iniciado por minúscula   | \$media | 89     | 2     |

Fonte: Elaborada pelos autores (2026).

O número de erros acompanha o número de ocorrências do lexema inválido. `Not` e `Media1` aparecem cinco vezes no programa, na declaração e nos usos, e cada ocorrência é reportada;

`$media` aparece duas vezes, na declaração e na chamada. Como o analisador não para no primeiro erro, todos são encontrados em uma única execução. Em `Media1`, a mensagem traz dois motivos ao mesmo tempo, o dígito e a terminação que não é letra minúscula. Em `05`, a mensagem sugere o uso de `_Z_`.

## 8 CONFERÊNCIA DAS REGRAS E OBSERVAÇÕES PARA A ETAPA 2

Além dos testes manuais, o repositório traz duas verificações automáticas. O *script* `test_regras.py`, documentado em `docs/conferencia_regras.md`, compara cada regra do enunciado com o comportamento do analisador, usando os casos citados no próprio enunciado, e confere o resultado do exemplo do professor. O resultado registrado é que todas as regras conferem. O *script* `test_simbolos.py` confere a tabela de símbolos gerada para o exemplo do professor e termina com a palavra `ok` quando passa.

A conferência também listou situações que não são erros do analisador léxico, mas que vão exigir cuidado na gramática da Etapa 2. A primeira é o número negativo escrito com sinal de menos, como `-5`, que passa como `MENOS` seguido de `NUMERO`, porque o léxico não tem como distinguir o menos de uma subtração do menos de um número negativo; cabe à gramática recusar o menos unário. A segunda é `3.5`, que passa como `NUMERO`, `PONTO` e `NUMERO`, já que o ponto é o fim de comando, e que a gramática vai recusar porque nada pode vir depois do ponto na mesma linha. A terceira é a regra de um comando por linha, que não é léxica, mas pode ser conferida depois, porque cada *token* guarda a linha em que apareceu. A quarta é `3x4` escrito sem espaços, que gera erro léxico, pois a multiplicação é uma letra e precisa de espaço em volta, assim como `EXP` e `RESTO`. A quinta é `Senao` com inicial maiúscula, que é um identificador e não a palavra reservada.

O erro de digitação encontrado no teste\_3 pertence à mesma família da quinta observação. Esses casos indicam que a Etapa 2 vai precisar de mensagens de erro sintático claras, capazes de avisar o programador quando um identificador muito parecido com uma palavra reservada aparece no lugar dela.

## 9 CONSIDERAÇÕES FINAIS

A Etapa 1 resultou em um analisador léxico para Elgol que cumpre as regras do enunciado e foi verificado de três formas: pela conferência automática das regras, pela instalação e execução em dois sistemas operacionais e por uma bateria de programas escritos

pelo grupo, corretos e com erros. Os quatro programas corretos somaram 356 *tokens* sem nenhum erro léxico, e os cinco programas com erros tiveram todos os problemas apontados com linha, coluna e motivo.

Do ponto de vista técnico, a principal lição foi que, em uma linguagem com regras léxicas incomuns, a expressão regular não deve carregar sozinha a validação. Deixar o padrão genérico e decidir no código Python permitiu mensagens de erro precisas e um único erro por lexema, ao custo de um pouco mais de código. A ordem de verificação das palavras reservadas, por sua vez, foi o que impediu que `MIgual` virasse nome de variável.

O trabalho de documentação também mostrou seu valor. As capturas de tela da instalação registraram um problema no arquivo de exemplo mínimo, e a revisão dos programas de teste encontrou um erro de digitação em uma palavra reservada. Os dois foram corrigidos no repositório. A próxima etapa do projeto é o analisador sintático, que deve usar o módulo `ply.yacc` e levar em conta as observações reunidas na Seção 8.

## REFERÊNCIAS

AHO, Alfred V. et al. **Compiladores**: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

BEAZLEY, David M. **ply 3.11**. [S. l.]: Python Package Index, 15 fev. 2018. Disponível em: <https://pypi.org/project/ply/3.11/>. Acesso em: 6 out. 2026.

BEAZLEY, David M. **ply**: Python Lex-Yacc. [S. l.]: GitHub, 2025. Repositório arquivado pelo autor em 21 dez. 2025. Disponível em: <https://github.com/dabeaz/ply>. Acesso em: 6 out. 2026.

CANONICAL. **How to set up a development environment for Python on Ubuntu**. [S. l.]: Canonical, [20--]. Disponível em: <https://documentation.ubuntu.com/ubuntu-for-developers/howto/python-setup/>. Acesso em: 6 out. 2026.

DEBIAN. **Details of package python3-venv in trixie**. [S. l.]: Debian, [20--]. Disponível em: <https://packages.debian.org/trixie/python3-venv>. Acesso em: 6 out. 2026.

KAYSER, Henrique Marques et al. **compiladores-elgol**: projeto da disciplina de Compiladores. [S. l.]: GitHub, 2026. Disponível em: <https://github.com/hmkayser-ops/compiladores-elgol>. Acesso em: 6 out. 2026.

## APÊNDICE A - CÓDIGO-FONTE DOS PROGRAMAS DE TESTE CORRETOS

Este apêndice reproduz, sem alterações, os quatro programas Elgol corretos da pasta `elgol` do repositório. A numeração das linhas foi acrescentada para facilitar a comparação com as colunas LINHA das Figuras 14 a 17.

### A.1 Arquivo teste\_1.elg

```

1  * Teste 1: Sistema de verificacao de aprovacao de aluno
2  * Palavras reservadas: elgio, DECIMAL, _Z_, se, entao, senao, MIgual,
   inicio, fim
3  * Operadores: =, +, /, ., ,, (, ), $
4
5  DECIMAL $Media (DECIMAL NotaUm, DECIMAL NotaDois) .
6  inicio .
7      DECIMAL Soma .
8      Soma = NotaUm + NotaDois .
9      elgio = Soma / 2 .
10 fim .
11
12 inicio .
13     DECIMAL NotaUm .
14     DECIMAL NotaDois .
15     DECIMAL Resultado .
16     DECIMAL Meta .
17
18     NotaUm = 5 .
19     NotaDois = 7 .
20     Meta = 6 .
21
22     Resultado = $Media (NotaUm, NotaDois) .
23
24     se Resultado MIgual Meta .
25     entao .
26     inicio .
27         Resultado = 1 .
28     fim .
29     senao .
30     inicio .
31         Resultado = _Z_ .
32     fim .
33 fim .

```

## A.2 Arquivo teste\_2.elg

```

1  * Teste 2: Sistema de desconto em compras
2  * Palavras reservadas: elgio, DECIMAL, EXP, RESTO, se, entao, senao,
   menor, maior, inicio, fim, x
3  * Operadores: =, /, ., (, ), $
4
5  DECIMAL $Desconto (DECIMAL Valor) .
6  inicio .
7      DECIMAL Fator .
8      Fator = 3 EXP 2 .
9      elgio = (Valor x Fator) / 10 . * 10% de desconto
10 fim .
11
12 inicio .
13     DECIMAL ValorCompra .
14     DECIMAL Limite .
15     DECIMAL Sobra .
16
17     ValorCompra = 130 .
18     Limite = 100 .
19     Sobra = ValorCompra RESTO 10 .
20
21     se Limite menor ValorCompra .
22     entao .
23     inicio .
24         se ValorCompra maior Limite .
25         entao .
26         inicio .
27             ValorCompra = $Desconto (ValorCompra) .
28         fim .
29     fim .
30     senao .
31     inicio .
32         ValorCompra = ValorCompra .
33     fim .
34 fim .

```

### A.3 Arquivo teste\_3.elg

```
1  * Teste 3: Calculo de fatorial e iteracoes de contagem
2  * Palavras reservadas: elgio, DECIMAL, enquanto, para, igual, maior,
   migual, inicio, fim, x
3  * Operadores: =, -, +, ., (, ), $
4
5  DECIMAL $Fatorial (DECIMAL Valor) .
6  inicio .
7      DECIMAL Total .
8      Total = 1 .
9
10     enquanto Valor migual 1 .
11     inicio .
12         Total = Total x Valor .
13         Valor = Valor - 1 .
14     fim .
15     elgio = Total .
16 fim .
17
18 inicio .
19     DECIMAL Numero .
20     DECIMAL Resultado .
21     DECIMAL Passo .
22
23     Numero = 5 .
24     Resultado = $Fatorial (Numero) .
25     Passo = 1 .
26
27     para Passo migual 10 .
28     inicio .
29         se Passo maior 5 .
30         entao .
31         inicio .
32             Resultado = Resultado + 1 .
33         fim .
34     fim .
35 fim .
```

#### A.4 Arquivo teste\_4.elg

```
1  * Teste 4: Validador de conta bancaria e saldo
2  * Palavras reservadas: elgio, DECIMAL, _Z_, _NEG_, diferente, se,
   entao, senao, inicio, fim
3  * Operadores: =, ., (, ), $
4
5  DECIMAL $VerificaConta (DECIMAL Saldo) .
6  inicio .
7      DECIMAL Status .
8
9      se Saldo diferente _Z_ .
10     entao .
11     inicio .
12         Status = 1 .
13     fim .
14     senao .
15     inicio .
16         Status = _Z_ .
17     fim .
18
19     elgio = Status .
20 fim .
21
22 inicio .
23     DECIMAL SaldoAtual .
24     DECIMAL Devedor .
25     DECIMAL Retorno .
26
27     SaldoAtual = 50 .
28     Devedor = _NEG_ SaldoAtual .
29
30     Retorno = $VerificaConta (Devedor) .
31 fim .
```